

RABBITMQ COMO MIDDLEWARE DE MENSAGERIA EM UMA APLICAÇÃO CLIENTE-SERVIDOR

2020004280 - Davi Hara de Godoy

2021018092 - Rafael M. da Silva Andrade

2020019184 - Vinicius R. Lourenço Marques

2020006874 - João Victor Fernandes

2020011524 - Guilherme Schmidt Fontes

SEMINÁRIO

Prof. Rafael Frinhani



INSTITUTO DE
**MATEMÁTICA E
COMPUTAÇÃO**

UNIFEI - Itajubá



RabbitMQ como Middleware de Mensageria em uma Aplicação Cliente-Servidor

1 Introdução

Em sistemas distribuídos, diferentes processos precisam se comunicar de forma organizada, eficiente e, em muitos casos, assíncrona. Essa comunicação pode se tornar complexa quando cliente e servidor estão em máquinas distintas, quando há variações no tempo de processamento ou quando é necessário desacoplar os componentes da aplicação [Tanenbaum & Steen \(2017\)](#).

Nesse contexto, middlewares de mensageria são utilizados para intermediar a troca de informações entre processos distribuídos. O RabbitMQ é uma tecnologia baseada em filas de mensagens que atua como *message broker*, permitindo que aplicações produtoras enviem mensagens para filas e que aplicações consumidoras as processem posteriormente [RabbitMQ \(2026b\)](#).

O objetivo deste seminário é apresentar o RabbitMQ como middleware de comunicação distribuída, abordando seu paradigma, arquitetura, funcionamento e cenários de uso. Além da parte conceitual, foi desenvolvido um protótipo em C#/.NET utilizando a biblioteca RabbitMQ.Client, no qual um cliente envia requisições a um servidor por meio do RabbitMQ [RabbitMQ \(2026c\)](#).

O protótipo implementa três operações principais: resposta a uma mensagem de texto, escrita em arquivo no servidor e realização de cálculos matemáticos. Também foi incluída uma fila assíncrona para demonstrar o envio de mensagens sem resposta imediata.

2 Fundamentação Conceitual

2.1 Paradigma de Mensageria

O RabbitMQ está associado ao paradigma de comunicação por mensageria, no qual os processos se comunicam por meio de mensagens intermediadas por um broker. Nesse modelo, o produtor da mensagem não precisa conhecer diretamente o consumidor, pois a comunicação ocorre por meio de filas [RabbitMQ \(2026b\)](#).

Esse paradigma favorece o desacoplamento entre os componentes do sistema, permitindo que produtores e consumidores operem de forma independente. Além disso, a comunicação pode ocorrer de maneira assíncrona, ou seja, o emissor pode enviar uma mensagem sem depender da execução imediata do receptor.

2.2 RabbitMQ

O RabbitMQ é um middleware de mensageria de código aberto que atua como *message broker*. Sua principal função é receber mensagens de aplicações produtoras, organizá-las em filas e entregá-las para aplicações consumidoras [RabbitMQ \(2026b\)](#). Ele é frequentemente utilizado em sistemas distribuídos, arquiteturas de microserviços, processamento de tarefas em segundo plano e integração entre aplicações.

No RabbitMQ, as mensagens podem ser roteadas por meio de exchanges, armazenadas em filas e consumidas por diferentes processos. Esse modelo permite maior flexibilidade na comunicação entre serviços e contribui para a escalabilidade do sistema [RabbitMQ \(2026a\)](#).

2.3 Principais Componentes

A arquitetura do RabbitMQ envolve alguns componentes fundamentais [RabbitMQ \(2026a\)](#):

- **Producer:** aplicação responsável por produzir e enviar mensagens.
- **Exchange:** componente responsável por receber mensagens e roteá-las para filas.
- **Queue:** fila onde as mensagens permanecem até serem consumidas.
- **Consumer:** aplicação responsável por consumir e processar mensagens.
- **Routing Key:** chave utilizada no roteamento das mensagens.

2.4 RPC sobre RabbitMQ

Embora o RabbitMQ seja baseado em mensageria assíncrona, também é possível implementar o padrão RPC (*Remote Procedure Call*) sobre filas. Nesse caso, o cliente envia uma requisição para uma fila e aguarda a resposta em uma fila temporária. Para relacionar a requisição com a resposta, são utilizados mecanismos como *CorrelationId* e *ReplyTo* [RabbitMQ \(2026d\)](#).

No protótipo desenvolvido, as operações principais utilizam esse padrão, pois o cliente precisa receber uma resposta após solicitar uma mensagem, uma escrita em arquivo ou um cálculo. Além disso, foi implementada uma fila assíncrona separada para demonstrar o envio de mensagens no modelo *publish-subscribe*.

3 Desenvolvimento do Protótipo

3.1 Arquitetura da Solução

O protótipo desenvolvido utiliza uma arquitetura cliente-servidor com o RabbitMQ atuando como broker de mensagens. O cliente foi implementado em C#/.NET e utiliza a biblioteca RabbitMQ.Client para enviar requisições ao RabbitMQ. O servidor, também desenvolvido em C#/.NET, consome as mensagens da fila, processa as operações solicitadas e retorna as respostas ao cliente [RabbitMQ \(2026c\)](#).

A fila principal utilizada para as operações com resposta é denominada `fila_rpc`. Para comunicação assíncrona sem resposta, foi implementado o padrão Publish-Subscribe utilizando um Exchange Fanout denominado `async_pubsub`, onde cada servidor cria automaticamente sua própria fila exclusiva e temporária que é vinculada ao exchange para receber mensagens broadcast.

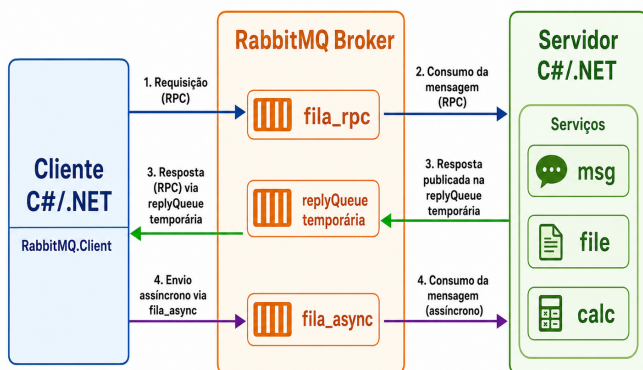


Figura 1: Arquitetura geral do protótipo com RabbitMQ.

3.2 Fluxo de Comunicação

O protótipo implementa dois padrões de comunicação distribuída sobre RabbitMQ, utilizando o protocolo AMQP (*Advanced Message Queuing Protocol*) como base da comunicação. O AMQP fornece uma padronização para a troca de mensagens entre aplicações distribuídas, definindo mecanismos de roteamento, filas, publicação e consumo de mensagens de forma interoperável e desacoplada [RabbitMQ \(2026a\)](#).

Para estruturar os dados trafegados entre cliente e servidor, foi utilizado o arquivo `RequestMessage.cs` como DTO (*Data Transfer Object*) e contrato de comunicação da aplicação. Essa estrutura contém os campos `Operation` e `Payload`, responsáveis por indicar a operação solicitada e os dados enviados na requisição, sendo utilizada tanto no padrão RPC quanto no Publish-Subscribe.

3.2.1 RPC (Remote Procedure Call)

O padrão RPC permite que o cliente execute operações remotas no servidor e aguarde uma resposta, simulando uma chamada de função local. O cliente serializa uma requisição em JSON e a publica na fila `fila_rpc`. Para receber a resposta, o

cliente cria uma fila temporária exclusiva e informa essa fila ao servidor por meio da propriedade `ReplyTo`.

O servidor consome a mensagem da fila, identifica a operação solicitada e executa o processamento correspondente. A propriedade `CorrelationId` é utilizada para associar a resposta recebida à requisição enviada, garantindo que o cliente correlacione corretamente a resposta mesmo em cenários de múltiplas requisições simultâneas. Após o processamento, o servidor publica a resposta na fila indicada pelo cliente.

```

1 public class RequestMessage
2 {
3     public string? Operation { get; set; }
4     public string? Payload { get; set; }
5 }
  
```

Figura 2: DTO estabelecido entre o cliente e o servidor.

3.2.2 Publish-Subscribe

O padrão Publish-Subscribe utiliza um Exchange Fanout denominado `async_pubsub` para realizar broadcasting de mensagens. Neste modelo, o cliente publica uma mensagem no exchange sem especificar um destinatário específico, e o RabbitMQ replica automaticamente essa mensagem para todas as filas vinculadas ao exchange.

Cada servidor cria uma fila exclusiva e temporária ao se conectar, vinculando-a ao exchange Fanout. Quando o cliente publica uma mensagem, todos os servidores conectados recebem uma cópia e processam independentemente, sem enviar resposta ao cliente.

3.3 Operações Implementadas

O servidor implementa três operações principais:

- **Mensagem de texto (msg):** o cliente envia uma mensagem e o servidor retorna uma resposta confirmando o recebimento.
- **Escrita em arquivo (file):** o cliente envia um conteúdo textual e o servidor grava esse conteúdo em um arquivo `file.txt`.
- **Cálculo matemático (calc):** o cliente envia uma operação e valores numéricos, e o servidor retorna o resultado calculado.

Além dessas operações, foi implementado o padrão Publish-Subscribe utilizando um Exchange Fanout denominado `async_pubsub`. Nesse caso, a mensagem é publicada no exchange e distribuída via broadcasting para todos os servidores conectados, sem que o cliente aguarde uma resposta.

3.4 Tecnologias Utilizadas

As principais tecnologias utilizadas foram:

- RabbitMQ como middleware de mensageria [RabbitMQ \(2026b\)](#);
- C#/.NET para implementação do cliente e servidor;
- RabbitMQ.Client como biblioteca de integração com o broker [RabbitMQ \(2026c\)](#);
- JSON como formato de representação das mensagens;
- Docker ou instalação local para execução do RabbitMQ;
- GitHub para armazenamento do código-fonte do projeto.

3.5 Trecho de Implementação

O cliente cria uma fila temporária de resposta, define as propriedades `CorrelationId` e `ReplyTo`, e publica a mensagem na fila RPC. Esse mecanismo permite que o servidor processe a requisição e envie a resposta correta ao cliente [RabbitMQ \(2026d\)](#).

```
var correlationId = Guid.NewGuid().ToString();
var replyQueue = _channel.QueueDeclare("", false, true, true).QueueName;

var props = _channel.CreateBasicProperties(); ;
props.CorrelationId = correlationId;
props.ReplyTo = replyQueue;

string? response = null;
```

Figura 3: Cliente configurando identificador único e fila temporária de resposta para o padrão Request-Reply.

O servidor processa a requisição, copia o `CorrelationId` original para as propriedades de resposta, e publica o resultado na fila especificada em `ReplyTo`. Esse mecanismo garante que o cliente identifique corretamente qual resposta corresponde à sua requisição através do `CorrelationId` [RabbitMQ \(2026d\)](#).

```
var replyProps = _channel.CreateBasicProperties();
replyProps.CorrelationId = ea.BasicProperties.CorrelationId;

var responseBytes = Encoding.UTF8.GetBytes(response);

channel.BasicPublish(
    exchange: "",
    routingKey: ea.BasicProperties.ReplyTo,
    basicProperties: replyProps,
    body: responseBytes
);
```

Figura 4: Trecho da implementação do servidor utilizando o `correlationId` enviado pelo cliente

4 Resultados e Evidências

A validação do protótipo foi realizada por meio da execução do cliente e do servidor conectados ao broker RabbitMQ. Durante os testes, cada elemento do sistema (cliente, servidor e broker) foi executado em máquinas separadas, porém conectadas dentro da mesma rede local. O cliente foi capaz de enviar requisições ao servidor utilizando o endereço do broker,

enquanto o servidor permaneceu aguardando mensagens na fila `rpc` e subscrito ao exchange `async_pubsub` para receber broadcasts.

4.1 Mensagem de Texto

Na primeira operação, o cliente enviou uma mensagem textual para o servidor. O servidor processou a requisição do tipo `msg` e retornou uma resposta confirmando o recebimento da mensagem.

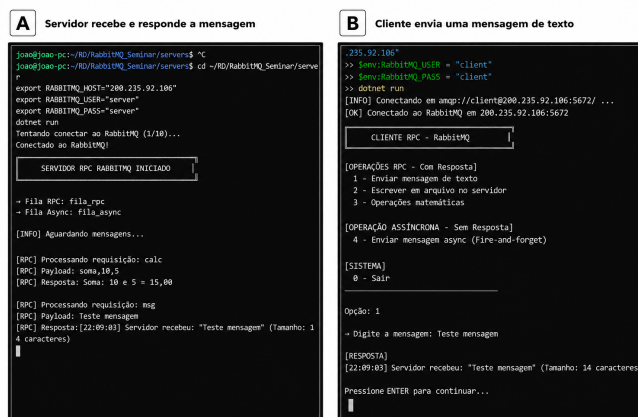


Figura 5: Evidências da troca de mensagens entre cliente e servidor.

4.2 Cálculo Matemático

Na operação de cálculo, o cliente enviou uma requisição contendo a operação matemática e os valores numéricos. O servidor processou a requisição do tipo `calc` e retornou o resultado ao cliente. Por exemplo, em um dos testes realizados, a soma de 10 e 5 retornou o resultado 15.

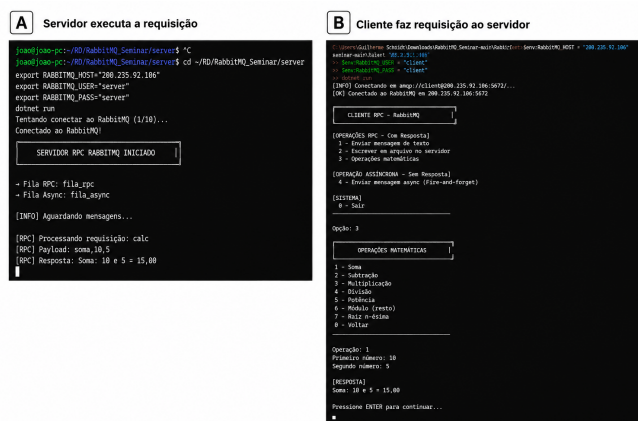


Figura 6: Evidências de execução de uma operação matemática.

4.3 Escrita em Arquivo

Na operação de escrita em arquivo, o cliente enviou um conteúdo textual para ser salvo no servidor. O servidor processou a requisição do tipo `file` e gravou as mensagens no arquivo `file.txt`.

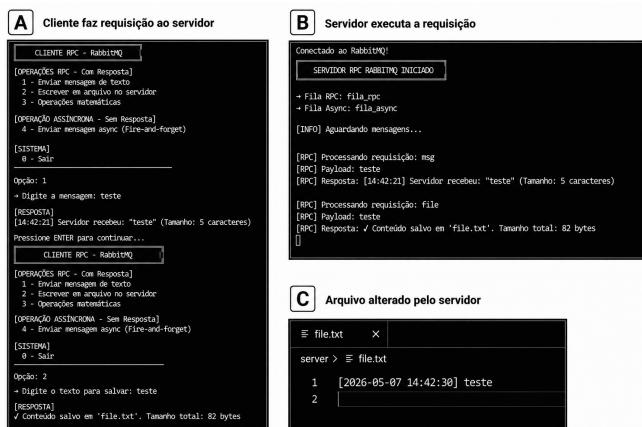


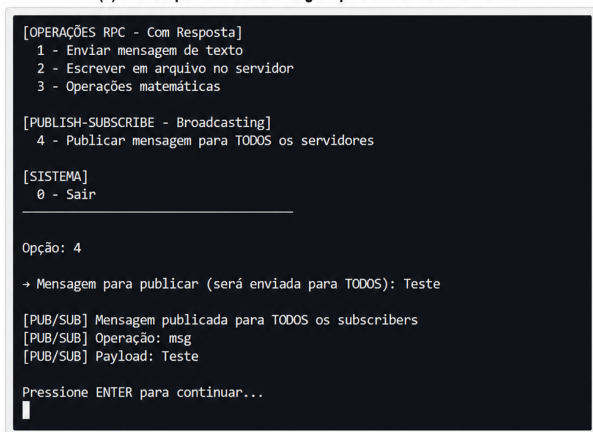
Figura 7: Evidências de execução do cliente, servidor e arquivo alterado.

4.4 Método Assíncrono

Na operação assíncrona, o cliente publicou uma mensagem no exchange `async_pubsub` utilizando o padrão Publish-Subscribe. Diferentemente das operações RPC, múltiplos servidores conectados ao exchange receberam simultaneamente a mesma mensagem via broadcasting. Cada servidor processou a mensagem de forma independente, sem enviar resposta ao cliente, demonstrando o comportamento do Exchange Fanout onde todos os subscribers recebem uma cópia da mensagem publicada.

Evidências de execução do padrão Publish/Subscribe

(a) Cliente publicando mensagem para todos os subscribers



(b) Servidor recebendo mensagem via broadcasting



Figura 8: Chamada assíncrona com broadcast.

5 Conclusões

O RabbitMQ demonstrou ser uma solução adequada para a comunicação entre processos distribuídos por meio de mensagens. Sua arquitetura baseada em filas permite desacoplar cliente e servidor, organizar o processamento das requisições e possibilitar diferentes modelos de comunicação RabbitMQ (2026b).

No protótipo desenvolvido, o RabbitMQ foi utilizado como broker entre uma aplicação cliente e uma aplicação servidora implementadas em C#/.NET. As operações principais foram implementadas utilizando o padrão RPC sobre RabbitMQ, permitindo que o cliente enviasse uma requisição e recebesse uma resposta processada pelo servidor. Além disso, o exchange `async_pubsub` demonstrou o uso do padrão Publish-Subscribe, permitindo broadcasting de mensagens para múltiplos servidores simultaneamente.

Entre as principais vantagens observadas estão o desacoplamento entre processos, a organização da comunicação por filas e a possibilidade de processamento assíncrono. Como limitações, destacam-se a dependência do broker, a necessidade de configuração adicional e a maior complexidade em comparação com uma comunicação direta entre cliente e servidor.

Portanto, o RabbitMQ é especialmente adequado para sistemas distribuídos que precisam integrar serviços, processar tarefas em segundo plano ou lidar com comunicação assíncrona entre componentes independentes.

6 Repositório do Projeto

O código-fonte do protótipo, os arquivos de configuração, os slides e as evidências complementares de execução estão disponíveis no repositório GitHub do projeto:

https://github.com/AndradeRafael41/RabbitMQ_Seminar/tree/pubsub

Referências

RabbitMQ (2026a). Amqp 0-9-1 model explained. <https://www.rabbitmq.com/tutorials/amqp-concepts>. Acesso em: 05 maio 2026.

RabbitMQ (2026b). Rabbitmq documentation. <https://www.rabbitmq.com/docs>. Acesso em: 05 maio 2026.

RabbitMQ (2026c). Rabbitmq .net/c# client api guide. <https://www.rabbitmq.com/client-libraries/dotnet-api-guide>. Acesso em: 05 maio 2026.

RabbitMQ (2026d). Rabbitmq tutorials. <https://www.rabbitmq.com/tutorials>. Acesso em: 05 maio 2026.

Tanenbaum, A. S. & Steen, M. v. (2017). *Distributed Systems*. Pearson, 3 edição.

